Introduction

This document profiles the two source datasets used in the Chicago and Dallas Food Inspections Data Pipeline — the Second Implementation deliverable for DAMG 7370 at Northeastern University. Both datasets are ingested as separate bronze-layer Delta tables and subsequently transformed and merged into a single, unified star schema dimensional model.

The two sources are:

- Chicago Food Inspections — published daily by the City of Chicago through the Chicago Open Data Portal. Each row represents one inspection event at a licensed food establishment in Chicago, IL.
- Dallas Food Inspections — published by the City of Dallas Open Data Portal. Each row represents one inspection event at a licensed restaurant in Dallas, TX. Unlike the Chicago dataset, violations are stored in wide/pivoted columns (up to 25 violation slots per row) and inspection results are provided as a numeric score rather than a text label.

The purpose of this profiling exercise is to document the completeness, consistency, and structural characteristics of both source tables, identify data quality issues that must be addressed in the ETL pipeline, and confirm that the planned transformations — including null handling, string standardisation, violation parsing, composite key construction, and SCD logic — are appropriate for each source.

Overview

Basic Information

Chicago Source — bronze_chicago_inspections

- Total Number of Records / Rows: ~250,000+ inspection events (growing daily)
- Total Number of Columns / Fields: 17 source columns
- Grain: One row per inspection event
- Primary Key: inspection_id (INT) — unique per inspection
- Natural Key for Establishments: license_# (INT)
- Violations Format: Single pipe-delimited free-text STRING in the violations column

Source Column	Source Type	Target Type	Nullable	Target Table
inspection_id	INT	STRING	No	dim_inspection
dba_name	STRING	STRING	Yes	dim_restaurant
aka_name	STRING	STRING	Yes	dim_restaurant
license_#	INT	STRING	Yes	dim_restaurant

Source Column	Source Type	Target Type	Nullable	Target Table
facility_type	STRING	STRING	Yes	dim_restaurant
risk	STRING	STRING	Yes	dim_inspection
address	STRING	STRING	Yes	dim_restaurant
city	STRING	STRING	Yes	dim_restaurant
state	STRING	STRING	Yes	dim_restaurant
zip	INT	STRING	Yes	dim_restaurant
inspection_date	DATE	DATE/INT	No	dim_date / fact
inspection_type	STRING	STRING	Yes	dim_inspection
results	STRING	INT	Yes	fact_food_inspection
violations	STRING	STRING	Yes	dim_violations / dim_comment
latitude	DOUBLE	DOUBLE	Yes	dim_restaurant
longitude	DOUBLE	DOUBLE	Yes	dim_restaurant
location	STRING	STRING	Yes	dim_restaurant

Dallas Source — bronze_dallas_inspections

- Total Number of Records / Rows: ~50,000+ inspection events (varies by export frequency)
- Total Number of Columns / Fields: ~80 columns (wide format — up to 25 violation slots × 3 sub-columns each, plus core fields)
- Grain: One row per inspection event (violations stored in wide/pivoted columns, not as a string)
- Primary Key: No native inspection ID — surrogate key generated in ETL as CONCAT('DAL-', sha2(name+date, 256).substr(1,12))
- Natural Key for Establishments: CONCAT(restaurant_name, '|', street_address) — no licence number available
- Violations Format: Wide columns — violation_description_N, violation_detail_N, violation_points_N for N = 1 to 25
- Inspection Result Format: Numeric score (INT) — unlike Chicago which uses a text label

Source Column	Source Type	Target Type	Nullable	Target Table
restaurant_name	STRING	STRING	No	dim_restaurant
street_address	STRING	STRING	Yes	dim_restaurant
zip_code	STRING	STRING	Yes	dim_restaurant (ZIP+4 → 5-digit)

Source Column	Source Type	Target Type	Nullable	Target Table
lat_long_location	STRING	DOUBLE×2	Yes	dim_restaurant (regex split)
inspection_date	DATE	DATE/INT	No	dim_date / fact
inspection_type	STRING	STRING	Yes	dim_inspection
inspection_score	INT	INT	Yes	fact_food_inspection (direct)
violation_description_N	STRING	STRING	Yes	dim_violations (N = 1–25)
violation_detail_N	STRING	STRING	Yes	dim_violations (N = 1–25)
violation_points_N	INT	INT	Yes	fact_food_inspection (N = 1–25)

Data Quality Assessment

Data quality issues for each source are documented separately below, followed by a cross-source comparison table. Each issue includes the affected column(s), estimated impact, and the specific action applied in the ETL pipeline.

Missing Values

Chicago — Missing Values

Column	Null Count (~)	Null %	Action Plan
aka_name	~112,000	~45%	Allow NULL in dim_restaurant.restaurant_name2 — field is optional
facility_type	~5,000	~2%	COALESCE(UPPER(TRIM(facility_type)), 'UNKNOWN')
risk	~12,000	~5%	COALESCE(TRIM(risk), 'Unknown') in dim_inspection
city	~500	<1%	COALESCE(UPPER(TRIM(city)), 'CHICAGO')
state	~500	<1%	COALESCE(UPPER(TRIM(state)), 'IL')
zip	~1,200	<1%	Cast to STRING; allow NULL — no safe default ZIP
latitude	~12,000	~5%	Allow NULL — geocoding not always performed
longitude	~12,000	~5%	Allow NULL — same batch as latitude NULLs
location	~12,000	~5%	Allow NULL — derived from lat/lng; null when both absent
violations	~30,000	~12%	Allow NULL — no violations cited; explode step produces 0 rows
results	~800	<1%	inspection_score defaults to 0 via CASE ELSE branch

Dallas — Missing Values

Column	Null Count (~)	Null %	Action Plan
street_address	~200	<1%	Allow NULL composite NK; route to 'unknown' establishment surrogate
zip_code	~400	<1%	Allow NULL in dim_restaurant; flag for review
lat_long_location	~3,000	~6%	Allow NULL for latitude and longitude after regex parse
inspection_type	~300	<1%	COALESCE(TRIM(inspection_type), 'Unknown')
inspection_score	~100	<1%	Allow NULL in fact; cannot derive a safe default for Dallas
violation_description_N	Sparse after slot 10	Varies by N	NULL slots produce no rows after unpivot — by design
violation_detail_N	Higher NULL rate than description_N	Varies	Allow NULL in dim_violations.violation_detail
violation_points_N	Sparse after slot 10	Varies	Default 0 points for NULL slots after unpivot

Duplicates

Chicago — Duplicates

Duplicate Type	Count (~)	% of Records	Action Plan
Exact full-row duplicates	~150	<0.1%	ROW_NUMBER() OVER (PARTITION BY inspection_id ORDER BY load_date DESC); keep rank=1
Duplicate inspection_id with differing data	~400	<0.2%	Retain latest version; log conflicts to audit error table
Same restaurant + same date (re-inspections)	~2,000	<1%	Expected and valid; retain all rows as distinct fact records

Dallas — Duplicates

Duplicate Type	Count (~)	% of Records	Action Plan
Exact full-row duplicates	~80	<0.2%	De-duplicate by (restaurant_name, street_address, inspection_date); keep latest
Same restaurant + date + different inspection_type	~150	<0.3%	Valid — Routine followed by Complaint on same day; retain both rows
Repeated violation slots with identical codes	~500	<1%	De-duplicate at dim_violations grain; retain one row per unique violation_code

Outliers

Chicago — Outliers

Column	Issue	Count (~)	Action Plan
zip	Values outside Chicago ZIP range 60601–60827	~200	Flag in audit; load as-is — may be adjacent suburbs
inspection_date	Dates before 2001 or after current date	~50	Reject row; log to error table
latitude	Values outside Chicago bounding box (41.6–42.0°N)	~30	Load as NULL; flag for GIS review
longitude	Values outside Chicago bounding box (−87.9–−87.5°W)	~30	Load as NULL; flag for GIS review
results	Values outside expected set (Pass/Fail/Conditions/No Entry)	~100	Default score = 0 via CASE ELSE; log unrecognised values
violations	Entries exceeding 5,000 characters	~500	No truncation; regex handles long strings gracefully

Dallas — Outliers

Column	Issue	Count (~)	Action Plan
zip_code	ZIP+4 format (e.g., '75201-1234') present alongside 5-digit ZIPs	~5,000	LEFT(zip_code, 5) to standardise; validated against Dallas ZIP range 75001–76000
lat_long_location	Values outside Dallas bounding box (32.6–33.0°N, −97.0–−96.5°W)	~50	Load as NULL after regex parse; flag for GIS review

Column	Issue	Count (~)	Action Plan
inspection_score	Scores below 0 or above 100	~20	Clamp to 0–100 range; log out-of-range values to error table
violation_points_N	Points values outside valid range (1–3)	~100	Default to NULL; severity derivation falls back to keyword-based CASE
inspection_date	Dates before 2008 (Dallas portal history start) or after current date	~30	Reject row; log to error table
restaurant_name	Names that are purely numeric or single-character (data entry errors)	~50	Flag for review; load as-is; do not default

Summary Statistics

Key statistical summaries and cardinality measurements are provided below for both source datasets, separated by numerical and categorical features.

Numerical Features

Chicago — Numerical Features

Column	Min	Max	Mean	Median	Std Dev	Notes
inspection_id	1	~2,398,012	~1,190,000	~1,185,000	~690,000	Sequentially assigned; no significant gaps
zip	60601	60827	60622	60614	±80	5-digit Chicago range; ~1,200 NULLs
latitude	41.645	42.022	41.856	41.857	±0.082	Chicago bounding box; ~12K NULLs
longitude	−87.913	−87.524	−87.678	−87.667	±0.091	Chicago bounding box; ~12K NULLs

Dallas — Numerical Features

Column	Min	Max	Mean	Median	Std Dev	Notes
inspection_score	0	100	~82	~85	±12	Actual numeric score; higher = better; ~100 NULLs

Column	Min	Max	Mean	Median	Std Dev	Notes
violation_points_N	0	3	~1.4	~1	±0.9	1=Minor, 2=Serious, 3=Critical; 0 for empty slots
latitude (parsed)	32.62	32.98	32.79	32.78	±0.08	Parsed from lat_long_location; ~3K NULLs
longitude (parsed)	-97.00	-96.56	-96.80	-96.79	±0.07	Parsed from lat_long_location; ~3K NULLs

Categorical Features

Chicago — Categorical Features

Column	Cardinality	Top Values (approx. %)	Notes
dba_name	~30,000	SUBWAY (~0.8%), MCDONALD'S (~0.7%), DUNKIN' (~0.5%)	Mixed case; UPPER(TRIM) required
aka_name	~15,000	~45% NULL; remainder mirrors dba_name or trade name	Optional; allow NULL
facility_type	~450	Restaurant (~55%), Grocery (~10%), School (~6%)	COALESCE to 'UNKNOWN' for ~2% NULLs
risk	6 values	Risk 1 (High) ~47%, Risk 2 (Med) ~30%, Risk 3 (Low) ~18%	All/N/A/Unknown ~5%
inspection_type	~20	Canvass ~55%, License ~15%, Complaint ~12%	UPPER(TRIM) to standardise
results	7 values	Pass ~50%, Fail ~23%, Pass w/ Conditions ~17%	Score derived via CASE expression
city	~25	CHICAGO ~99.8%; others are suburbs	COALESCE to 'CHICAGO' for NULLs
state	~4	IL ~99.9%, IN <0.1%	COALESCE to 'IL' for NULLs

Dallas — Categorical Features

Column	Cardinality	Top Values (approx. %)	Notes
restaurant_name	~8,000	Large chains appear frequently (e.g., MCDONALD'S, TACO BELL)	UPPER(TRIM) required; used as part of composite NK
inspection_type	~5	Routine (~65%), Re-Inspection (~20%), Complaint (~12%)	Mapped to standardised inspection_category in dim_inspection

Column	Cardinality	Top Values (approx. %)	Notes
violation_description_N	~150 unique codes	Coded violations e.g. '32.', '18.'; '*' prefix on critical codes	Regex extracts code; '*' indicates critical violation
zip_code	~200	75201, 75205, 75220 most common; ZIP+4 variants present	LEFT(zip_code,5) standardises to 5-digit format

Data Transformation

Transformations are described below for both source datasets, grouped by transformation category.

Chicago Transformations

- String standardisation — UPPER(TRIM()) applied to dba_name, aka_name, facility_type, address, city, state, and inspection_type to eliminate case and whitespace inconsistencies.
- Null coalescing — city → 'CHICAGO', state → 'IL', facility_type → 'UNKNOWN', risk → 'Unknown'. Geolocation NULLs are preserved.
- ZIP code normalisation — zip (INT) cast to STRING and truncated via LEFT(..., 5).
- Inspection score derivation — CASE expression on results: Pass = 100, Pass w/ Conditions = 80, Fail = 60, all other values = 0.
- Date dimension derivation — 12 date attributes generated from inspection_date using PySpark date functions (year, quarter, month, week, day-of-year, day-of-month, day-of-week, day-name, weekend flag).
- Violations string parsing — Split on pipe-and-digit pattern, regexp_extract for code and description, then split on '| Comments:' for inspector comment. Produces rows for dim_violations (deduplicated) and dim_comment (per inspection–violation pair).
- Surrogate key generation — All dimension PKs generated via monotonically_increasing_id().
- SCD Type 2 expiry — When restaurant_name, street_address, or facility_type change, old row is expired (effective_end_date = current_date(), is_current = FALSE) and a new row is inserted.

Dallas Transformations

- Composite natural key construction — No licence number in Dallas; establishment NK built as CONCAT(UPPER(TRIM(restaurant_name)), '|', UPPER(TRIM(street_address))).
- ZIP+4 normalisation — zip_code is a STRING that may contain ZIP+4 format; LEFT(zip_code, 5) standardises to 5-digit.
- Lat/long parsing from combined field — lat_long_location is a STRING in format '(lat, lng)'; latitude extracted via regexp_extract(lat_long_location, '\(([^,]+)', 1) and longitude via regexp_extract(lat_long_location, ',\s*([^)]+)', 1), both CAST to DOUBLE.

- Violation unpivot — Wide columns (violation_description_N, violation_detail_N, violation_points_N for N=1–25) are unpivoted (stacked) into a long format before being loaded into dim_violations and fact_food_inspection. NULL slots are dropped during unpivot.
- Violation severity derivation — CASE expression: points=3 OR 'Urgent'/'Critical' in description → 'Critical'; points=2 → 'Serious'; otherwise → 'Minor'.
- Inspection score → result text derivation — Dallas provides the numeric score directly; result_text is derived: score ≥ 90 → 'Pass', 80–89 → 'Pass w/ Conditions', 70–79 → 'Fail', < 70 → 'Fail'.
- Inspection type category alignment — Dallas types are mapped to standardised Chicago categories: Routine → 'Routine', Follow-up → 'Re-Inspection', Complaint → 'Complaint'.
- Synthetic inspection ID generation — CONCAT('DAL-', sha2(restaurant_name + inspection_date, 256).substr(1,12)) since Dallas has no native inspection identifier.
- Literal source_city injection — Literal 'Dallas' appended to all Dallas rows for all dimension and fact tables, enabling city-level filtering without a join.

Insights and Observations

The following cross-source patterns and dataset-specific anomalies were identified during profiling:

- The two datasets are structurally very different — Chicago uses a single free-text violations string (long/narrow format) while Dallas uses up to 75 wide violation columns (short/wide format). The ETL pipeline must apply fundamentally different parsing strategies for each, but the output is a unified dim_violations table.
- Dallas provides actual numeric inspection scores; Chicago provides text outcomes — This asymmetry is handled in the ETL by deriving scores from Chicago text labels (Pass=100, Fail=60, etc.) while passing Dallas scores through directly. Both populate the same inspection_score column in fact_food_inspection.
- Dallas has no licence number — The composite natural key (restaurant_name + street_address) is less stable than Chicago's licence number, making SCD Type 2 change detection more complex. Name-spelling changes can cause false new records.
- The lat_long_location field in Dallas is a combined STRING, not two separate columns — This requires regex parsing and introduces a higher failure risk than the separate latitude/longitude columns in Chicago. The ~6% NULL rate for Dallas geolocation is slightly higher than Chicago's ~5%.
- Canvass inspections dominate Chicago (~55%) while Routine inspections dominate Dallas (~65%) — These are conceptually equivalent inspection types; the inspection_category mapping aligns them for cross-city analysis.
- Dallas violation severity can be derived from both points (1–3) and keywords in the description — The keyword-based fallback ('Urgent', 'Critical', 'Serious') is important because violation_points_N is NULL for some slots, particularly in older records.

- Chicago has ~250K records vs Dallas ~50K — Approximately 5:1 ratio. Cross-city dashboards should normalise by establishment count or time period rather than using raw inspection counts to avoid misleading comparisons.

Conclusion and Recommendations

Both source datasets are suitable for loading into the unified star schema dimensional model, but each presents distinct data quality and structural challenges. The key cross-cutting findings and recommendations are:

- Implement source-aware ETL branching — The Chicago and Dallas pipelines should be separate notebooks that converge at the dimension and fact load steps. Sharing a single notebook would make the parsing logic fragile. Each source should write to staging tables before merging into the conformed dimensions.
- Handle the Dallas violation unpivot defensively — The wide-to-long transformation for up to 25 violation slots is the highest-risk step in the Dallas pipeline. Recommendation: validate that the unpivot produces a minimum of 1 and maximum of 25 rows per inspection, and log any inspection row that produces 0 violation rows (possible indicator of a parsing failure rather than a genuinely violation-free inspection).
- Cross-validate surrogate keys across both sources — dim_violations and dim_restaurant must not generate duplicate surrogate keys when Chicago and Dallas rows are merged. Recommendation: assign surrogate key ranges per source (e.g., Chicago 1–999999999, Dallas 1000000000+) or use a global monotonically_increasing_id() applied after both sources are unioned.
- Add a source_city filter column to all dimension and fact tables — The Literal 'Chicago' and 'Dallas' source_city columns are already planned; confirm they are applied consistently in all 6 target tables to enable clean city-level partitioning in BI dashboards.
- Resolve the inspection score scale difference before reporting — Chicago scores (100/80/60/0) are ordinal labels, not continuous measurements. Dallas scores (0–100) are continuous. Analysts must be warned that AVG(inspection_score) across both cities is not a meaningful metric. Recommendation: add a score_type column to fact_food_inspection with values 'derived' (Chicago) and 'actual' (Dallas).
- Profile Dallas violation_description_N columns more deeply before go-live — The '*' prefix on critical violation codes in Dallas is a data quality signal that must be handled reliably by the regex. Recommendation: run a full audit of all distinct values in violation_description_1 through violation_description_25 and confirm the regexp_extract pattern captures all code formats present in the data.